Week 9 · Putting the system together

Programming for Information Systems

Where should your decisions live?

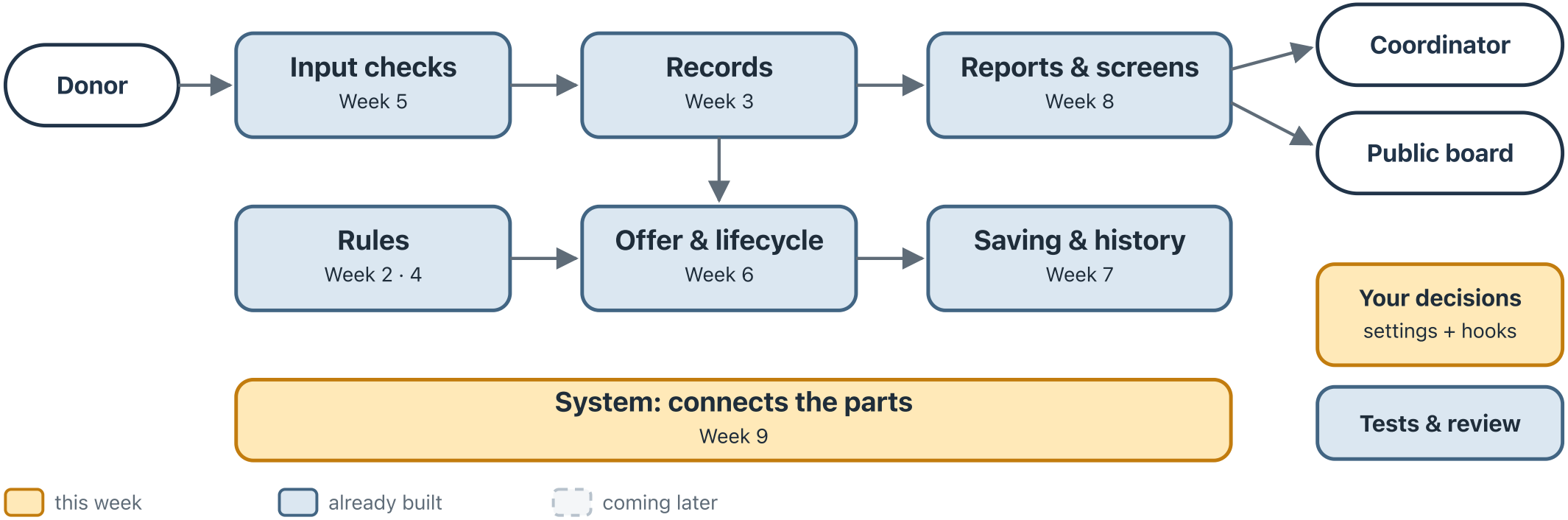
This week

Question	How do the parts fit together, and where should your decisions live?
Python	modules, `import`, `if __name__ == "__main__":`; run a file with `subprocess.run`
Principle 9	Separate what changes from what stays
You build	your own system with `bridge.System(MY_SETTINGS, hooks=...)`, then Final Level 1

Your system, by Week 10

Your FoodBridge system

Week 9: connect every part and add your decisions



The idea: what changes and what stays

- On a phone you change the ringtone, language and brightness in **Settings**.
- You never open the phone to rewire it.
- **Machinery** stays: checking input, records, saving, screens.
- **Decisions** change: the team minimum, the public fields, your own rules.

> Decisions live in two places only: **settings** and **hooks**.

Python foundations

Modules, programs and the guard

A module is a `.py` file

```
import math
print(math.sqrt(49))            # 7.0

import foodbridge as bridge     # the setup cell does this
print(bridge.rules.pickup_label(30, False))  # hold
print(bridge.records.total_portions(bridge.sample_records()))  # 100
```

- `import` loads a module. `module.name` = the name inside it.
- `as bridge` gives the module a short name.

The map of the parts

Module	Week	Job
`rules.py`	2	Pickup label and urgency
`records.py`	3	Records, IDs, totals
`validation.py`	5	Check input at the door
`models.py`	6	The Offer and its lifecycle
`storage.py`	7	Save safely, keep history
`reports.py`, `menu.py`	8	Views for different people
`system.py`	9	Connect the parts, plug in your hooks

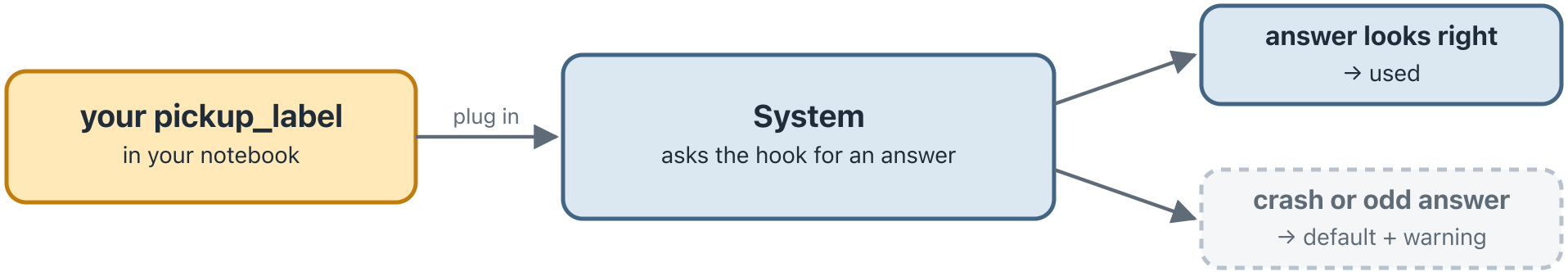
Two places for your decisions

```
print(len(bridge.DEFAULTS), "settings")  # 15 settings
print(bridge.DEFAULTS["TEAM_PICKUP_MIN_PORTIONS"])  # 20
print(bridge.HOOKS)
# {'pickup_label': 2, 'urgency': 1, 'clean_offer': 1,
#  'can_move': 2, 'public_row': 1}
```

- **Settings** (`bridge.DEFAULTS`): values. Change them in `MY_SETTINGS`.
- **Hooks** (`bridge.HOOKS`): functions you may replace, with their number of inputs.

How `System` connects the parts

Hooks: your function inside the real system



1. Complete settings: defaults + your changes (misspelt names ignored, with ⚠️).
2. One default function per hook; yours replace them.
3. Ask the hook for each answer. Crash or odd answer → default + ⚠️ once.

Safe failure, in practice

```
def shouting_label(portions, approved):
    return "TEAM!"

test_system = bridge.System({"TEAM_PICKUP_MIN_PORTION": 25},
                            hooks={"pickup_label": shouting_label})
```

```
⚠ Setting ignored: TEAM_PICKUP_MIN_PORTION is not a known
  setting. Did you mean TEAM_PICKUP_MIN_PORTIONS?
⚠ Your pickup_label(30, True) did not work: it returned
  'TEAM!' ... The system used the default rule instead.
```

Run a file in a fresh process

```
import subprocess
import sys

with open("hello.py", "w") as file:
    file.write('print("Hello from hello.py")\n')

result = subprocess.run([sys.executable, "hello.py"],
                        capture_output=True, text=True)
print(result.stdout, result.returncode)
```

- A fresh process knows **only** what is in its own file.
- `stdout` = output · `stderr` = errors · return code `0` = no error.

Importing or running: `__name__`

```python
# shop.py
def price_with_tax(price):
    return round(price * 1.10, 2)

if __name__ == "__main__":        # only when shop.py is run
    print("Demo:", price_with_tax(100))
```

- Run `shop.py` → `__name__` is `"__main__"` → the demo runs.
- `main.py` does `import shop` → `__name__` is `"shop"` → no demo.

Run the kit's own tests

```
kit_tests = subprocess.run([sys.executable, "run_tests.py"],
                           cwd="foodbridge_kit",
                           capture_output=True, text=True)
print(kit_tests.stdout.splitlines()[-1])    # ... passed, 0 failed
```

- `cwd=` starts the program inside the kit folder.
- These tests use the **default** settings: they protect the machinery.

Principle 9 · Separate what changes from what stays

- A decision written inside the machinery is **hidden**: nobody can find it or change it safely.
- Two copies of one rule will **disagree** after the first change.
- Keep decisions in settings and hooks. Let the machinery **ask** for them.

> When you review AI code, ask: *is a decision buried here?*

⚖️ Good or risky?

Version A (from an AI assistant)

```
def report_line(record):
    if record["portions"] >= 20:
        team = "team"
    else:
        team = "standard"
    return f"{record['id']} · {team}"

def web_public_row(record):
    row = dict(record)
    del row["contact"]
    return row
```

Version B

```
def report_line(system, offer):
    label = system.label(offer)
    return f"{offer.id} · {label}"

def web_public_row(system, record):
    fields = system.settings["PUBLIC_FIELDS"]
    return bridge.reports.public_row(record, fields)
```

Repair: move the decision into a setting

```
def web_row(record, settings):
    if record["hours_left"] <= settings["URGENT_HOURS"]:
        badge = "URGENT"
    else:
        badge = ""
    shown = {}
    for field in settings["PUBLIC_FIELDS"]:
        shown[field] = record.get(field, "")
    return badge, shown
```

- The check tries **two** different settings. Buried numbers fail one of them.

You build: your own system

```
MY_SETTINGS = {"SYSTEM_NAME": "Battambang Food Share",
               "TEAM_PICKUP_MIN_PORTIONS": 25, ...}   # every setting
TEAM_PICKUP_MIN_PORTIONS = MY_SETTINGS["TEAM_PICKUP_MIN_PORTIONS"]
MY_HOOKS = {"pickup_label": pickup_label}   # optional

my_system = bridge.System(MY_SETTINGS, hooks=MY_HOOKS)
my_system.run_day(show=False)
my_system.verify()
```

- Read constants **from** `MY_SETTINGS`: each number is written once.
- `verify()`: kit checks all ✅; *Your system* lists your decisions and any ⚠️.

🧭 Your decisions so far

Week	Setting or hook	My choice	Alternative	Why	Who is affected
2	`TEAM_PICKUP_MIN_PORTIONS`				
5	`MAX_PORTIONS`				
8	`PUBLIC_FIELDS`				
9	my hooks				

- Final: at least **3** decisions; at least one about **privacy** (`PUBLIC_FIELDS`) or **input checks**.

Start the Final project now

1. Open the Final from the course home page; **File → Save a copy in Drive**.
2. **Level 1 · Run it:** setup → `bridge.System()` → `run_day()` → `verify()`: all ✅.
3. **Level 2 · Make it yours:** paste your `MY_SETTINGS`, hooks and decision table.

> From now on, the Final is the one place where your decisions live.

This week's tasks

- ⭐ **Core:** Session 1 concepts with predictions; P1–P4; Good or risky; your system assembled and verified; decisions table.
- ⭐ **In class:** Final project Level 1 done, Level 2 started.
- ⭐⭐ **Stretch:** `daily_summary.py` program (S1); settings in `my_settings.py` with the text menu.
- No research task this week.

Next week · Verify, hand over, present

How do you know your system works, and could someone run it **without you**? Tests for your own decisions · a classmate runs your system · a 3-minute presentation.